GoF Design Patterns and Architecture

Purpose.

Distinguish formal GoF design patterns such as Adapter, Strategy, Builder, and Factory Method from architectural and design roles such as Service, Orchestrator, Provider, Policy, DI, and Exception, and clearly define the responsibility and application scope of each concept.

1. Key Distinctions

An Architecture Style is a high-level structural approach that defines how Layers, Boundaries, Components, and dependencies are organized across the system. Representative examples include Layered, Hexagonal, Clean, Onion, and Event-Driven architectures.

A Design Pattern is a reusable solution structure for recurring software design problems. GoF patterns are classified as Creational, Structural, and Behavioral.

An Architecture / Design Role describes the responsibility assigned to a particular layer or object in a system. A role is not itself a GoF pattern, and multiple design patterns may be applied within a single role.

An Architecture / Design Approach determines the perspectives and principles used to analyze and model a system. Examples include DDD, OOD, and Responsibility-Driven Design. MVC2 is classified separately as a Presentation/Web Architecture Pattern that describes HTTP request handling and response coordination.

2. Software Architecture & Design Classification

Classification	Representative Examples	Key Question	Design Outcome
Architecture Style	Layered, Hexagonal, Clean, Onion, Event-Driven	How should Components, Layers, Boundaries, and dependencies be structurally organized across the system?	Layer structure, Port/Adapter boundaries, Dependency Rule, Event flow
Design Approach	DDD, OOD, Responsibility-Driven Design, Contract/API-First Design	What should the design center on, and which design mindset should be used?	Domain Model, object responsibilities, collaboration relationships, Interface/API Contract
Design Role	Service, Orchestrator, Provider, Policy, DI, Exception	What responsibility does this object or Component have in the system?	Clear separation of responsibilities and collaboration relationships
Design Pattern	Adapter, Strategy, Builder, Factory Method, Abstract Factory	Which proven structure should solve a recurring object/class design problem?	Object creation, composition, collaboration, and interchangeable algorithm structures

3. Architecture Style

An Architecture Style is a high-level design approach that determines the overall structure and dependency relationships of a system. Unlike GoF Design Patterns, which primarily address recurring object- and class-level design problems,

Architecture Styles address system-level concerns such as layers, boundaries, dependency direction, deployment, and communication structures.

Architecture Style	Key Question	Primary Characteristics
Layered Architecture	How should responsibilities be separated by layer?	Separates responsibilities into layers such as Presentation, Application, Domain/Core, and Infrastructure
Hexagonal Architecture	How should the Core be isolated from external technologies?	Places Ports at the Core boundary and connects external Frameworks/APIs through Adapters
Clean Architecture	How should dependencies point toward core policies?	Applies the Dependency Rule so inner Business Rules do not depend on outer Frameworks
Onion Architecture	How should dependency layers be arranged around the Domain?	Places the Domain Model at the center, with Application and Infrastructure in outer layers
Event-Driven Architecture	How should state changes and asynchronous processing connect components?	Uses Event publishing/subscription to reduce coupling between components and construct asynchronous flows
Modular Monolith	How should module boundaries be maintained within a single deployment unit?	Deploys as one application while maintaining clear responsibility and dependency boundaries between functional modules

4. Architecture / Design Approach

While GoF Design Patterns provide solution structures for recurring class- and object-level design problems, an Architecture / Design Approach is a higher-level approach that determines broader system structure, responsibility boundaries, domain modeling methods, and request-processing methods.

Category	Approach	Key Question	Primary Responsibility / Scope
Presentation Architecture	MVC2	Who receives HTTP requests, and how are processing results connected to responses?	Separates Controller-centered request handling, Model/Application invocation, and View or Response generation
Domain Design Approach	DDD	How should complex business Domains and rules be accurately represented in code?	Domain-centered design using Ubiquitous Language, Bounded Context, Entity, Value Object, Aggregate, Repository, Domain Service, and related concepts

Difference between MVC2 and DDD. MVC2 primarily describes the Presentation/Web request-processing structure, whereas DDD is a design approach for analyzing and modeling the Domain. They are not competing concepts and can be applied together within the same system.

Classification note. MVC2 and DDD are not included among the 23 GoF design patterns. MVC2 is classified as a Web/Presentation Architecture Pattern, while DDD is classified separately as a Domain-centered Design Approach.

5. Architecture / Design Role

Classification	Role / Concept	Key Question	Primary Responsibility
Application Role	Service	Which Use Case does it provide?	Provides the Application boundary and exposes functionality for external callers
Workflow Role	Orchestrator	In what order should	Coordinates the call order of

		execution occur?	multiple components and the overall Workflow
Provisioning Role	Provider	Which object or configuration should be provided?	Selects and provides an object, policy, or configuration appropriate to the conditions
Rule Role	Policy	Which rules should be applied?	Defines decision criteria, constraints, and selection rules by domain and situation
Dependency Management	DI	Where are objects connected?	Creates, selects, and injects dependencies externally and constructs the Object Graph
Error Handling	Exception	What failure occurred?	Represents error types and communicates failure information across layers

6. GoF Design Patterns

Classification	Pattern	Key Question	Primary Responsibility
Structural	Adapter	How should different Interfaces be connected?	Converts an existing/external Interface into the Interface expected by the Core
Behavioral	Strategy	Which algorithm should be used for execution?	Encapsulates algorithms with the same purpose so they can be interchanged
Creational	Builder	How should a complex object be constructed step by step?	Separates the construction process of a composite object into multiple configuration steps
Creational	Factory Method	How should concrete object creation be delegated?	Defines an object-creation Interface and separates the decision of which concrete type to instantiate
Creational	Abstract Factory	How should families of related objects be created consistently?	Abstracts the creation of families of mutually related implementation objects

Note: The term 'Factory,' commonly used in practice, is not itself the name of a single GoF pattern. The GoF catalog defines Factory Method and Abstract Factory separately.

7. Distinctions in One Sentence

Service: Exposes Use Cases to external callers.

Orchestrator: Coordinates the execution order of multiple components.

Adapter: Connects otherwise incompatible Interfaces.

Provider: Selects and provides the object or configuration currently required.

Policy: Defines applicable rules, decision criteria, and constraints.

Strategy: Makes the algorithm used to perform a selected objective interchangeable.

Builder: Constructs a complex object step by step.

Factory Method / Abstract Factory: Abstracts the responsibility for creating concrete objects or families of related objects.

DI: Connects and injects dependencies externally across the system.

Exception: Represents and communicates the type and meaning of a failure.

8. Commonly Confused Relationships

Policy vs Strategy

Policy defines rules for deciding 'what to select or allow,' while Strategy determines 'which algorithm to use' to perform the selected objective. Example: if RetrievalPolicy selects HYBRID, HybridRetriever performs the actual retrieval.

Builder vs Factory

Factory focuses on encapsulating object creation type and creation responsibility, while Builder focuses on constructing a complex object step by step when many settings and parts are required.

Provider vs Factory

Provider is concerned with 'providing what is needed.' It may return an existing instance or use a Factory to create a new object. Therefore, Provider is a role, whereas Factory Method and Abstract Factory are creational patterns.

Builder vs Orchestrator

Builder configures objects before execution. Orchestrator uses already prepared objects to execute a workflow.

Service vs Orchestrator

Service is the Use Case boundary exposed by the Application to external callers, while Orchestrator coordinates complex execution flows within that Use Case. For a simple Use Case, the Service may coordinate the flow directly, making a separate Orchestrator unnecessary.

DI vs Builder

DI configures dependency relationships and the Object Graph at the system level. Builder focuses on the step-by-step construction of a specific complex object.

Exception vs Policy

Exception represents 'what failed.' Policy may define criteria for 'how to respond,' such as retrying, stopping, or applying a Fallback after a failure.

9. Application Principles

- Do not choose a pattern name first and then create classes around it. Apply patterns only where actual variability or responsibility separation is required.
- The Core should not directly depend on external Frameworks such as LangChain, LangGraph, or FastAPI.
- Separate Policy from Strategy so that changes to decision rules and changes to algorithms have minimal impact on each other.
- Do not introduce Provider, Builder, and Factory redundantly. If object creation is simple, DI alone may be sufficient.
- A Service and an Orchestrator are not always both necessary. Separate an Orchestrator when a Use Case becomes complex enough that coordinating multiple components becomes a distinct responsibility.
- Exceptions should express the meaning of a failure, while transport-specific concerns such as HTTP status codes should be mapped at the Interface layer.

10. References

- Refactoring.Guru, Design Patterns Catalog — Classification of GoF-related patterns into Creational, Structural, and Behavioral categories, with explanations of patterns such as Adapter, Builder, and Strategy.
- Martin Fowler, Patterns of Enterprise Application Architecture — Service Layer: Defines an application's boundary and available operations through a service layer.
- Martin Fowler, Layering Principles — Layering principles including low coupling between layers, separation of concerns, and independence from higher-level consumers.